

Autonomous Software Engineering Operating System

Abstract

Software engineering is a complex, multi-stage activity involving requirements analysis, repository understanding, design, implementation, debugging, testing, code review, maintenance, and continuous improvement. Traditional AI-assisted development tools generally focus on isolated activities such as code completion, question answering, or generating patches. Recent research has demonstrated that Large Language Model (LLM)-based agents can interact with software repositories, use external tools, execute programs, and solve increasingly complex software engineering tasks. However, existing systems still face significant challenges involving repository-level reasoning, long-horizon planning, reliable execution, uncertainty management, verification, recovery from failed actions, persistent project memory, resource management, and safe autonomous operation.

This project proposes an **Autonomous Software Engineering Operating System (ASEOS)**, a desktop-first intelligent engineering environment capable of maintaining a persistent model of a software project and autonomously performing multi-step engineering activities. Instead of treating an agent as a simple prompt-response system, the proposed platform maintains an **Engineering World Model** representing source files, symbols, dependencies, tests, issues, architecture, decisions, technical debt, historical actions, and execution outcomes. Based on this representation, a Decision Engine determines appropriate actions and dynamically creates or assigns specialized workers according to required capabilities.

The system will support repository analysis, task interpretation, dynamic planning, code modification, experimentation, testing, debugging, code review, Git worktree isolation, risk assessment, human approval, persistent memory, execution replay, and software-health analysis. The model-provider layer will remain independent of any particular model and communicate through an **LLM API Call**, allowing a future choice of provider or model without redesigning the system.

The proposed architecture combines a Tauri desktop application, React and TypeScript interface, Python-based agent runtime, repository intelligence using Tree-sitter/ripgrep/LSP, isolated execution using Docker, persistent project state using SQLite, optional vector retrieval using Chroma, policy-based human-in-the-loop controls, and OpenTelemetry-based observability. The project will be evaluated using task success, test-pass rate, regression rate, recovery rate, human-intervention rate, cost, latency, and reliability-oriented experiments.

1. Introduction

1.1 Background

The software development process traditionally depends on human developers who continuously switch between several activities: understanding requirements, exploring repositories, locating relevant

implementation components, designing modifications, writing code, executing tests, investigating failures, reviewing changes, and maintaining system quality. These activities require not only programming knowledge but also contextual reasoning over large and evolving software systems.

The emergence of Large Language Models has substantially changed software development assistance. LLMs can generate code, explain programs, identify bugs, generate tests, and interact with tools. However, a conventional LLM typically operates as a stateless text generator unless it is augmented with tools, memory, execution environments, and mechanisms for iterative interaction with the external world.

The ReAct paradigm demonstrated that reasoning and action can be interleaved, enabling language models to use external tools and update their plans based on observations. Toolformer further demonstrated that language models can learn when to call external APIs and incorporate tool outputs into subsequent reasoning.

These ideas have led to the development of LLM-based agents. Frameworks such as AutoGen demonstrate multi-agent collaboration, while systems such as ChatDev and MetaGPT explore software development through multiple specialized AI roles.

At the same time, research has increasingly moved from generating isolated code snippets toward solving complete repository-level software engineering problems. SWE-bench introduced a benchmark in which an agent receives a real GitHub issue and repository and must produce a modification that resolves the problem. The benchmark showed that repository-level software engineering requires much more than code generation because agents must understand multiple files, interact with execution environments, reason about dependencies, and validate their changes.

SWE-agent extended this direction by studying an agent-computer interface specifically designed for software engineering, enabling agents to navigate repositories, edit files, and execute tests. OpenHands similarly provides an environment in which software-development agents can write code, interact with command-line environments, browse the web, use sandboxes, and coordinate agents.

Despite these advances, autonomous software engineering remains an open problem. Agents can generate incorrect changes, misunderstand requirements, enter inefficient execution loops, overuse tools, fail to recover from errors, or claim task completion without sufficient evidence. A 2024 survey of LLM-based software engineering agents identifies challenges across requirements engineering, code generation, decision-making, software design, testing, and maintenance.

The proposed project addresses this broader problem by treating software development as a **persistent engineering environment** rather than a sequence of independent prompts.

1.2 Motivation

Current AI coding tools frequently optimize for local assistance:

“Generate this function.”

“Explain this error.”

“Modify this file.”

However, real software engineering frequently involves tasks such as:

“The checkout system occasionally fails in production. Find the root cause, fix it, write regression tests, and make sure existing functionality still works.”

Such a task may require:

1. Understanding repository architecture.
2. Finding relevant services and modules.
3. Inspecting dependencies and call relationships.
4. Examining historical changes.
5. Reproducing the failure.
6. Developing competing hypotheses.
7. Designing experiments.
8. Modifying multiple files.
9. Running targeted and regression tests.
10. Diagnosing failures.
11. Revising the implementation.
12. Reviewing the final patch.
13. Assessing risk.
14. Presenting evidence of completion.

A useful engineering agent therefore needs a persistent understanding of the project, not merely access to a long context window.

This motivates the proposed system.

2. Literature Review

2.1 LLMs as Reasoning and Acting Systems

ReAct introduced an approach in which language-model reasoning and external actions are interleaved. Rather than reasoning entirely before acting, the system can observe the environment, act, receive observations, and alter its subsequent plan. This approach is particularly relevant to software engineering because software tasks inherently involve iterative observation and execution.

Toolformer explored another important component: deciding when and how an LLM should call external tools. This established the foundation for systems in which models are not limited to textual reasoning but can invoke APIs and incorporate their outputs into later decisions.

For the proposed project, these works motivate the **observe** → **decide** → **act** → **observe** interaction cycle.

2.2 Multi-Agent Systems

AutoGen demonstrated programmable LLM-agent conversations involving combinations of agents, tools, and human interaction.

ChatDev investigated software development using multiple specialized LLM-based agents involved in design, coding, and testing. Its findings demonstrated that communication between specialized agents can contribute to software-development workflows.

MetaGPT extended role-based collaboration by attempting to encode software-development workflows into multi-agent collaboration.

AgentCoder further explored specialized agents for programming, test design, and test execution, allowing code generation to be iteratively refined using execution feedback.

These studies show that specialization and collaboration can be beneficial. However, a limitation for the proposed project is that simply replicating static roles such as “Developer Agent,” “Tester Agent,” and “Reviewer Agent” does not fully address dynamic software-engineering environments.

Therefore, the proposed architecture does not make fixed agent roles the primary abstraction. Instead, it uses a **capability-driven Agent Fabric**, where workers are created or configured according to the current engineering objective.

2.3 Repository-Level Software Engineering

SWE-bench significantly changed the evaluation of AI coding systems by using real GitHub issues and repositories rather than isolated programming exercises. A task can require modifications across multiple functions, classes, and files and may require execution of the project to determine whether the proposed change actually works.

CodeAgent investigated tool-integrated repository-level coding and reported that giving agents tools for information retrieval, code navigation, implementation, and testing can substantially improve performance compared with simpler code-generation approaches.

SWE-agent emphasized that agent-computer interfaces strongly influence software-engineering performance. Its custom interface was designed around repository navigation, file editing, execution, and testing rather than generic text interaction.

OpenHands similarly demonstrates the importance of an execution environment, command-line interaction, sandboxing, tool use, and repository manipulation for software-development agents.

These studies motivate the proposed **Engineering Environment Layer**, in which code, Git, tests, terminals, runtime environments, and project metadata become first-class inputs to the agent.

2.4 Agentless Approaches and the Importance of Simplicity

An important counterpoint is presented by Agentless. The work demonstrated that a comparatively simple process involving localization, repair, and patch validation could perform surprisingly well on SWE-bench Lite, raising the question of whether complex autonomous agents are always necessary.

This finding is important for the proposed project.

The objective is therefore not:

“Use as many agents as possible.”

Instead, the objective is:

Select the simplest execution strategy that provides sufficient reliability for the current engineering problem.

For a small bug, a single worker may be enough.

For a distributed incident, the system may require several specialized workers.

This gives the proposed architecture an adaptive rather than permanently multi-agent structure.

2.5 Memory and Persistent Agent State

Generative Agents demonstrated an architecture in which an agent maintains records of experiences, forms higher-level reflections, and retrieves memories while planning future actions.

For software engineering, persistent memory can be particularly valuable because projects accumulate knowledge over time. Examples include:

- architectural decisions;
- recurring bugs;
- repository-specific conventions;
- known fragile components;
- successful debugging procedures;
- previous failed attempts;
- test limitations;
- deployment constraints.

The proposed system therefore introduces multiple forms of engineering memory rather than relying only on conversation history.

2.6 Human-in-the-Loop Systems

Fully autonomous agents present reliability and safety problems. Recent research on human-agent collaboration identifies hallucination, complex task handling, reliability, and safety as important limitations of fully autonomous systems and highlights human intervention as a mechanism for improving reliability.

LangGraph also provides practical mechanisms for interrupting execution, saving state, obtaining human decisions, and resuming workflows.

The proposed system extends this idea through **risk-aware approval**, where the level of human involvement is determined by the potential consequences of an action.

For example:

Action	Example	Approval
Low risk	Read source file	None
Low risk	Run unit tests	None
Medium risk	Modify source files	Optional / policy-based
High risk	Install dependency	Approval
High risk	Delete files	Approval
Critical	Push/merge/deploy	Explicit approval

2.7 Evaluation of AI Agents

Recent research increasingly emphasizes that evaluating agent systems requires more than measuring answer quality. A 2026 survey of LLM-agent evaluation highlights planning, tool use, application-specific benchmarks, robustness, safety, and cost-efficiency as important evaluation dimensions.

SWE-bench has been an influential benchmark for software-engineering agents, but evaluation methodology itself is evolving. In February 2026, OpenAI reported that SWE-bench Verified had become increasingly contaminated by training exposure and recommended newer, less-contaminated evaluation settings such as SWE-bench Pro for frontier coding evaluations.

This motivates the project to develop its own **task-oriented engineering evaluation suite**, while using public benchmarks only as secondary evaluation references.

3. Research Gap

The literature demonstrates strong progress in:

- tool-using LLMs;
- multi-agent collaboration;
- repository-level coding;
- autonomous software-development environments;
- sandboxed execution;
- memory;
- human-in-the-loop workflows.

However, the following capabilities remain insufficiently integrated into a single coherent engineering operating system:

3.1 Persistent Engineering World Model

Most coding agents retrieve repository context as required, but the proposed system maintains a structured representation of the project containing code entities, dependencies, tests, issues, architecture, decisions, risk, historical work, and current state.

3.2 Dynamic Agent Formation

Instead of fixed roles, the system selects workers according to required capabilities, allowing the organization structure to adapt to the task.

3.3 Uncertainty-Driven Investigation

The system explicitly represents multiple hypotheses and can conduct experiments to determine which explanation is most consistent with available evidence.

3.4 Evidence-Based Completion

Task completion should be determined by tests, builds, static analysis, runtime evidence, and review—not solely by the LLM declaring success.

3.5 Autonomous Recovery

Failed actions should trigger diagnosis, checkpoint restoration, replanning, and controlled retry instead of simply ending the task.

3.6 Engineering Experience Learning

The system should learn procedural knowledge from previous successful and unsuccessful engineering activities.

3.7 Continuous Software Health Management

The system should not only respond to user instructions but also identify technical debt, flaky tests, architectural problems, dependency issues, and other engineering-health concerns.

4. Problem Statement

Existing AI coding assistants and LLM-based agents primarily provide assistance at the level of code generation, conversational interaction, or task-specific automation. Although recent research has improved repository-level reasoning and tool use, autonomous software engineering remains difficult because an agent must operate for long periods in a changing environment, maintain project context, reason across multiple artifacts, select suitable actions, respond to execution feedback, recover from failure, and establish evidence that its changes are correct.

The core problem addressed by this project is therefore:

How can an AI-driven software engineering environment autonomously understand a continuously evolving software project, decide what actions are required to achieve an engineering objective, dynamically allocate appropriate computational and agent resources, execute and verify changes in an isolated environment, recover from failures, maintain persistent engineering memory, and involve humans according to risk while preserving reliability and control?

The proposed system aims to solve this problem through a persistent Engineering World Model, adaptive decision engine, capability-based agent fabric, tool execution layer, verification engine, risk policy engine, memory system, and human-in-the-loop controls.

5. Aim of the Project

The primary aim is to design and implement a desktop-first **Autonomous Software Engineering Operating System** capable of managing complex software-development tasks from problem interpretation through verified implementation.

The project aims to move beyond simple AI code generation toward **autonomous engineering workflows with evidence, memory, recovery, and adaptive decision-making**.

6. Objectives

6.1 Primary Objectives

1. Develop a desktop environment for interacting with AI engineering capabilities.
2. Build a persistent representation of a software repository.
3. Automatically analyze repository structure, symbols, dependencies, and tests.
4. Interpret natural-language engineering requirements.
5. Generate adaptive plans rather than fixed workflows.
6. Dynamically assign or create specialized workers based on capabilities.
7. Allow workers to inspect and modify repositories.
8. Execute generated code inside isolated environments.
9. Run deterministic tests, builds, linters, and static checks.
10. Diagnose failures and autonomously retry or replan.
11. Maintain short-term, episodic, procedural, and project-level memory.
12. Introduce risk-aware human approval.
13. Record execution traces and allow replay of previous tasks.
14. Maintain an engineering-health view of the project.
15. Evaluate system reliability using controlled software-engineering tasks.

6.2 Secondary Objectives

The system should also support:

- Git worktrees;
- pull-request preparation;
- dependency-impact analysis;
- regression detection;
- architecture visualization;
- task prioritization;
- software-health recommendations;
- model-provider abstraction;
- resource-aware execution;
- benchmark and evaluation dashboards.

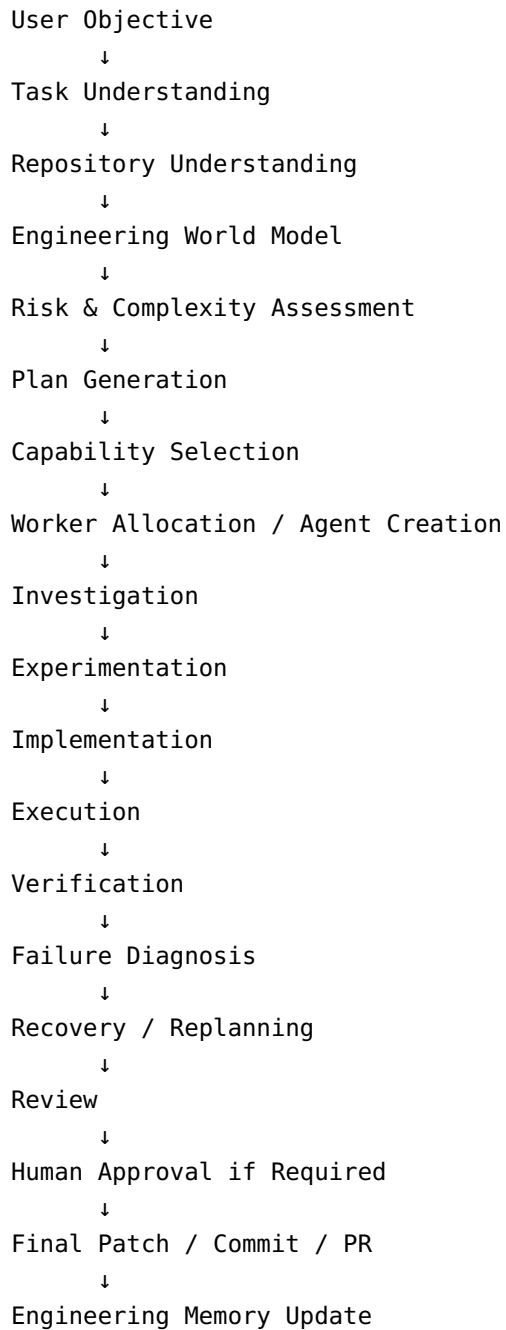
7. Proposed System

The proposed platform is a **local-first AI-native software engineering environment**.

The user does not interact with independent AI chatbots. Instead, the user provides an engineering objective such as:

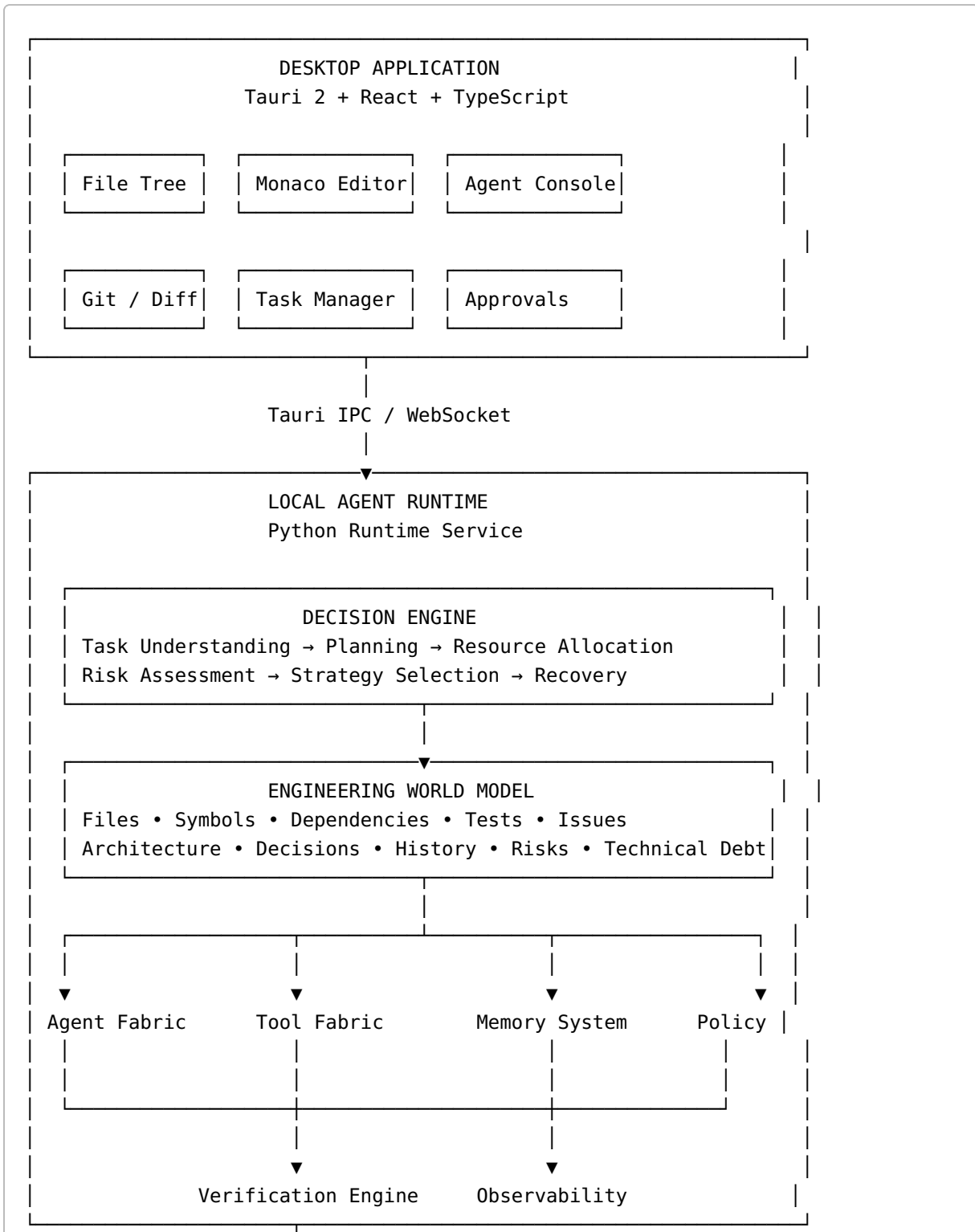
“Investigate why users occasionally receive duplicate notifications and fix the problem without breaking existing behavior.”

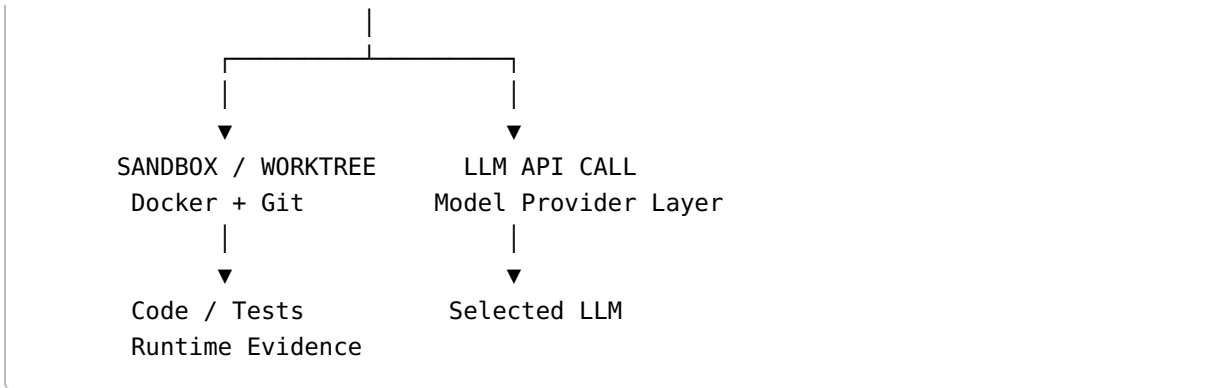
The platform then performs:



8. System Architecture

8.1 High-Level Architecture





9. Architectural Layers

9.1 Presentation Layer

The desktop application will be built using **Tauri 2**, React, and TypeScript.

Tauri allows a web-based frontend to operate as a desktop application while integrating with native functionality through Rust. Its architecture uses the operating system's webview rather than shipping a complete browser engine, which helps keep the application comparatively lightweight.

The interface will contain:

- repository explorer;
- code editor;
- integrated terminal;
- task panel;
- agent execution timeline;
- live tool calls;
- diff viewer;
- approval queue;
- project-health dashboard;
- architecture graph;
- memory explorer;
- execution replay;
- Git state.

9.2 Agent Runtime

A local Python service will operate as the engineering runtime.

Potential components include:

- FastAPI;
- Pydantic;
- asynchronous task execution;
- structured tool calls;
- state management;
- agent execution;
- planning;
- policy evaluation.

FastAPI provides a modern Python framework for building high-performance APIs using Python type hints, making it suitable for the runtime/control interface.

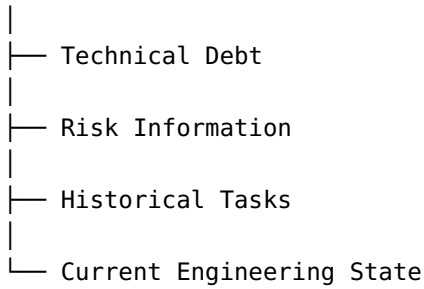
The runtime can communicate with the Tauri application using local IPC, WebSockets, or a localhost service.

10. Engineering World Model

The Engineering World Model is the central abstraction of the system.

Instead of treating the repository as a collection of text files, the platform maintains structured entities such as:

```
Project
|
├── Repository
|   ├── Files
|   ├── Directories
|   ├── Symbols
|   ├── Functions
|   ├── Classes
|   └── Modules
|
├── Dependencies
|
├── Tests
|
├── Services
|
├── APIs
|
├── Issues
|
└── Architecture Decisions
```



Relationships can include:

```
Function A
|
|— calls → Function B
|— tested-by → Test X
|— imported-by → Module C
|— modified-in → Commit Z
```

This enables impact analysis such as:

“Changing this database model affects four services and eleven tests.”

11. Codebase Intelligence

The Codebase Intelligence subsystem will use multiple complementary techniques.

11.1 Lexical Search

ripgrep or an equivalent high-performance search system will be used for rapid exact or regular-expression searches.

11.2 Syntax and Structure

Tree-sitter will provide syntax-aware parsing and incremental syntax trees. Tree-sitter is designed to parse programming languages efficiently and remain useful even when source code contains syntax errors.

This can be used to extract:

- functions;
- classes;
- imports;
- variables;
- declarations;

- APIs;
- syntax relationships.

11.3 Semantic Code Navigation

The system may integrate with the **Language Server Protocol (LSP)** to obtain language-aware information such as definitions, references, diagnostics, and type information.

11.4 Semantic Retrieval

An optional vector layer will allow semantically similar code and engineering documentation to be retrieved.

Chroma supports storing embeddings and metadata and performing dense, sparse, and hybrid retrieval, making it suitable for local experimentation with semantic code search.

12. Decision Engine

The Decision Engine is responsible for deciding:

- what should be done;
- which information is missing;
- which worker capabilities are required;
- whether an experiment should be performed;
- which tool should be used;
- whether execution should continue;
- whether the task should be replanned;
- whether human intervention is required.

The system does not assume that every task requires the same workflow.

For example:

```
Task A:  
"Rename this function."
```

```
→ Single worker  
→ Modify  
→ Test  
→ Done
```

Whereas:

Task B:

"Investigate intermittent payment failures."

- Incident investigator
- Repository analyst
- Database analyst
- Test engineer
- Multiple experiments
- Runtime verification
- Review

This makes the system **adaptive rather than statically multi-agent**.

13. Agent Fabric

Workers are created based on capabilities.

Possible capabilities include:

- repository analysis;
- debugging;
- code generation;
- test generation;
- security analysis;
- database analysis;
- performance analysis;
- frontend analysis;
- backend analysis;
- documentation;
- code review;
- incident investigation.

An agent instance may contain:

Agent

- └─ Capability Set
- └─ Objective
- └─ Permissions
- └─ Model Policy
- └─ Tool Access
- └─ Memory Context
- └─ Token/Time Budget
- └─ Current Hypotheses

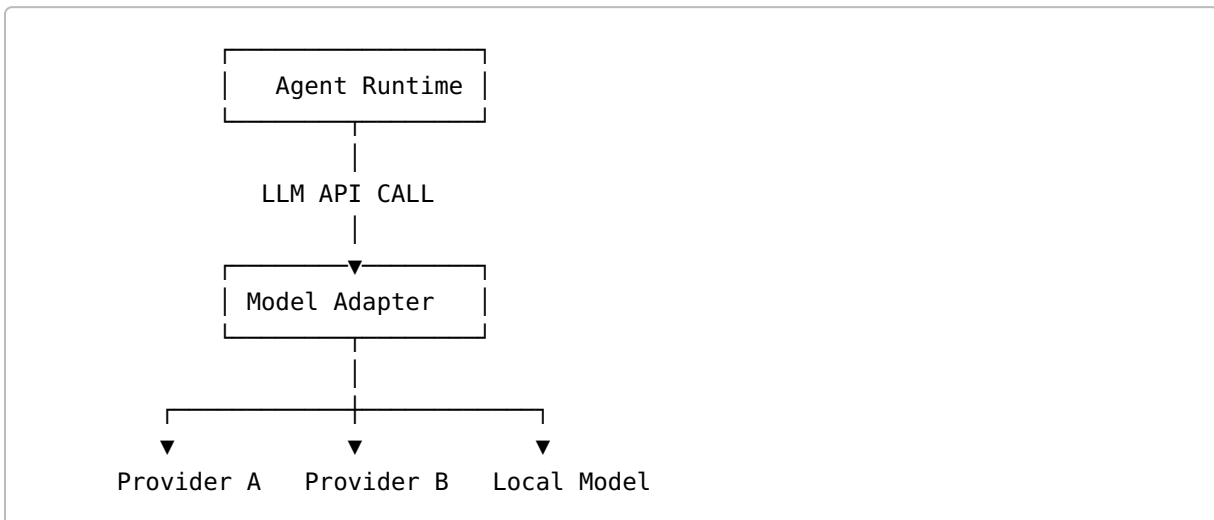
- └ Evidence
- └ Checkpoints
- └ Execution History

This turns agents into runtime entities rather than merely prompt templates.

14. LLM API Layer

The project will **not lock the system to a particular LLM**.

The architecture will contain:



The final model or provider will be selected later based on:

- coding ability;
- context-window requirements;
- tool-calling support;
- latency;
- cost;
- rate limits;
- privacy;
- availability.

The rest of the system should remain independent of that decision.

15. Tool Fabric

The tool layer provides controlled access to the software environment.

Core tools

```
list_files
read_file
search_code
find_symbol
find_references
apply_patch
create_file
delete_file
git_status
git_diff
git_log
git_worktree
run_command
run_test
run_build
run_linter
run_typecheck
inspect_process
inspect_logs
```

Optional external tools may include:

```
web_search
documentation_search
API_call
database_query
package_registry
issue_tracker
GitHub
```

Every tool invocation should generate an event containing:

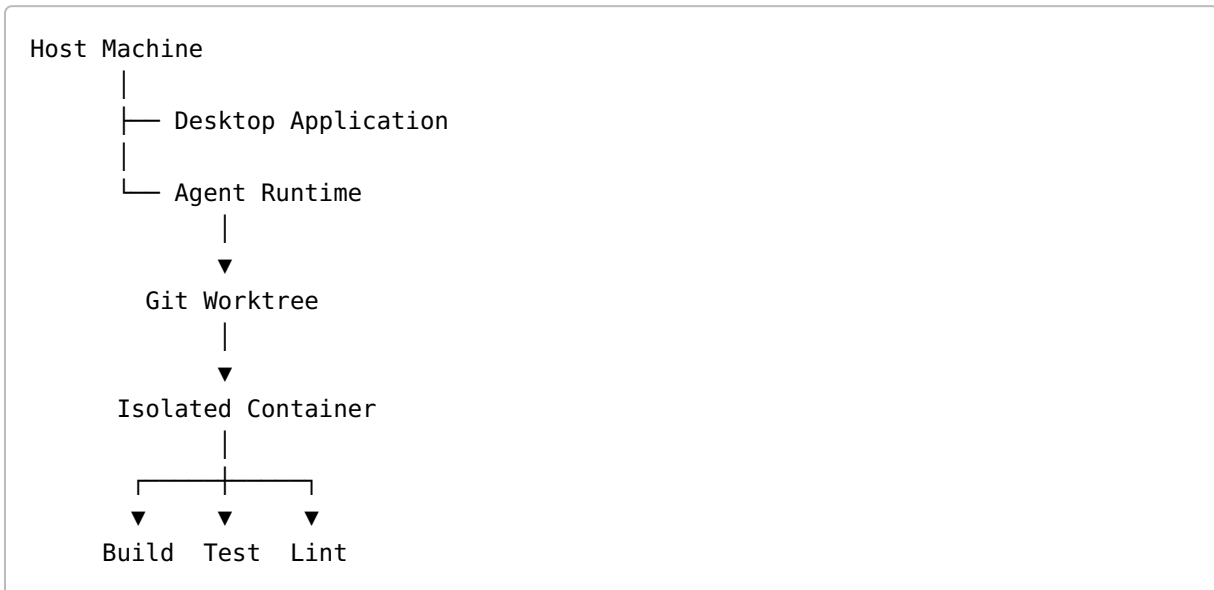
```
Task ID
Agent ID
Tool
Arguments
Timestamp
Duration
Status
Result
Risk Level
```

16. Execution Sandbox

Generated or modified code should not automatically execute with unrestricted host privileges.

Docker provides isolated processes with separate filesystem, networking, and process environments, making containers suitable for creating controlled execution environments.

The platform should use:



The isolation policy should restrict:

- filesystem access;
- network access where necessary;
- process privileges;
- package installation;
- resource consumption;
- execution duration.

17. Git Worktree and Checkpoint System

Every non-trivial autonomous change should occur inside an isolated Git worktree.

Example:

```
main
|
├── agent/task-001
|
├── agent/task-002
|
└── agent/experiment-003
```

This supports:

- isolation;
- parallel experiments;
- rollback;
- comparison;
- conflict reduction;
- safe human review.

The system can create checkpoints before significant modifications.

If an experiment damages the working state:

```
Current State
  ↓
Failure
  ↓
Restore Checkpoint
  ↓
Change Strategy
  ↓
Retry
```

18. Uncertainty and Experiment Engine

A major component of the proposed system is the ability to reason using competing hypotheses.

Example:

```
Observed:
Duplicate notifications

Hypothesis 1:
Message queue retry
```

Hypothesis 2:
Database transaction failure

Hypothesis 3:
Race condition

Hypothesis 4:
Duplicate API request

Rather than selecting one explanation immediately, the system can design experiments:

```
Hypothesis
  ↓
Experiment
  ↓
Observation
  ↓
Evidence
  ↓
Confidence Update
  ↓
Next Experiment
```

Possible experiments include:

- targeted tests;
- logging instrumentation;
- concurrency tests;
- Git history analysis;
- database queries;
- request replay;
- performance profiling;
- dependency inspection.

This creates a **scientific debugging loop** instead of pure guess-and-edit behavior.

19. Verification Engine

The system must not consider an engineering task complete merely because an LLM claims success.

Completion requires evidence from deterministic systems.

Verification may include:

```
Unit Tests
Integration Tests
Regression Tests
Type Checking
Linting
Build
Static Analysis
Security Scans
Runtime Checks
API Checks
```

For example:

```
LLM:
"Bug fixed."

System:
FAIL – regression test failed.

    ↓

Diagnose

    ↓

Modify

    ↓

Run tests again

    ↓

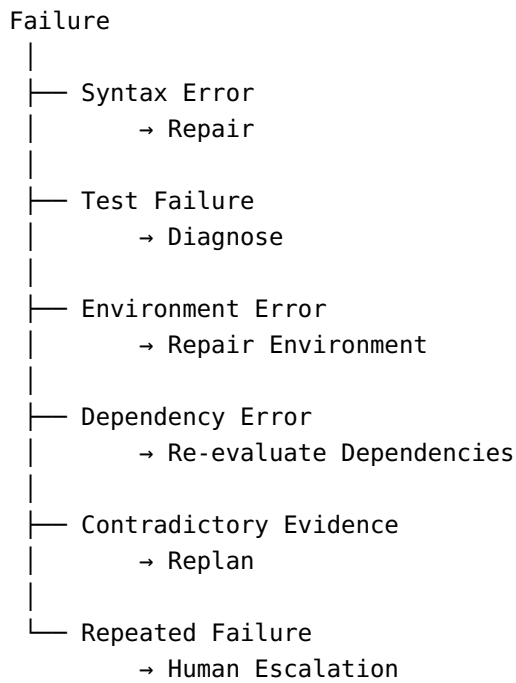
PASS
```

The verification layer therefore acts as a boundary between probabilistic reasoning and deterministic evidence.

20. Adaptive Recovery System

Failures are classified rather than simply retried.

Example:



The system should maintain:

- retry limits;
- budget limits;
- iteration limits;
- rollback points;
- failure history.

21. Memory Architecture

The memory subsystem will contain multiple layers.

21.1 Working Memory

Temporary context for the current task.

21.2 Episodic Memory

Historical engineering activities.

Example:

Task:
Fix database timeout

Attempts:
1 → failed
2 → identified connection pool issue
3 → fixed

21.3 Procedural Memory

Reusable engineering strategies.

Example:

"When debugging flaky asynchronous tests:
run isolated test repeatedly before modifying implementation."

21.4 Project Memory

Persistent knowledge specific to the repository:

- architecture decisions;
- coding conventions;
- deployment assumptions;
- known issues;
- fragile modules.

21.5 Agent Performance Memory

Historical worker performance:

Debugging	89%
SQL	83%
Frontend	74%
Security	68%

This information can influence future worker selection.

22. Risk and Human-in-the-Loop System

The system uses policy-driven controls.

A simplified risk score could be:

```
Risk =  
Blast Radius  
× Reversibility  
× Production Impact  
× Uncertainty  
× Historical Failure Rate
```

The score determines autonomy.

Example:

```
Risk < 0.20  
→ Autonomous  
  
0.20–0.50  
→ Autonomous + notification  
  
0.50–0.75  
→ Human approval required  
  
> 0.75  
→ Mandatory human control
```

Possible approval modes include:

- Approve action;
- Edit action;
- Reject action;
- Approve plan;
- Take over.

LangGraph's persistence and interrupt mechanisms provide a practical basis for pausing and resuming such workflows.

23. Observability

All important system events will be recorded.

The system should be able to answer:

What did the agent do?

Why did it do it?

Which model was used?

Which tools were called?

How much time was spent?

Where did failure occur?

Which checkpoint was restored?

When did human intervention occur?

OpenTelemetry is a vendor-neutral observability framework for generating, collecting, and exporting telemetry such as traces, metrics, and logs.

A trace could therefore look like:

```
Task
├─ Planning
├─ Repository Search
├─ Hypothesis Generation
├─ Experiment 1
├─ Experiment 2
├─ Code Modification
├─ Test
│   └─ FAIL
├─ Diagnosis
├─ Code Modification
├─ Test
│   └─ PASS
├─ Review
└─ Human Approval
```

24. Replay System

The platform should store sufficient information to reconstruct completed tasks.

Replay should display:

```
Initial State
  ↓
Observation
  ↓
Decision
  ↓
Tool Call
  ↓
Result
  ↓
Next Decision
  ↓
Modification
  ↓
Verification
```

This allows:

- debugging agent behavior;
- analyzing failures;
- comparing strategies;
- evaluating model changes;
- reproducing autonomous runs.

LangGraph persistence similarly supports checkpoints and time-travel/replay-oriented workflows.

25. Continuous Engineering Health

The system should not only respond to explicit tasks.

A project-health subsystem can periodically inspect:

```
Technical Debt
Code Complexity
Duplicated Code
Flaky Tests
```

Outdated Dependencies
Architecture Violations
Security Issues
Dead Code
Documentation Drift
Performance Regressions
Test Coverage Gaps

It can produce:

ENGINEERING HEALTH

Critical

- Payment retry race condition

High

- Authentication coupling
- 5 flaky integration tests

Medium

- Dependency drift
- Duplicate validation logic

Low

- Documentation inconsistencies

This converts the system from a reactive coding assistant into a **continuous software maintenance platform**.

26. Data Storage Architecture

For a desktop-first implementation, the initial architecture can use local storage.

SQLite

SQLite is a self-contained, serverless, zero-configuration transactional database stored in a single file, making it particularly suitable for local application state.

SQLite can store:

- tasks;
- execution records;
- agent state;

- project metadata;
- checkpoints;
- policies;
- memory metadata;
- evaluation results.

Chroma

Optional semantic memory can be maintained using Chroma for:

- code embeddings;
- project documentation;
- procedural memories;
- historical task retrieval.

Redis

Redis may be introduced when distributed execution or cloud-scale asynchronous workloads become necessary. Redis supports use cases involving in-memory state, caching, messaging, and streaming.

Redis is therefore **optional in the desktop MVP rather than mandatory**.

27. Functional Modules

Module	Main Responsibility
Desktop UI	User interaction and engineering workspace
Repository Manager	Repository discovery and workspace handling
Code Intelligence	AST, symbols, references, dependency analysis
Engineering World Model	Persistent project representation
Task Manager	User goals and engineering tasks
Decision Engine	Planning and strategic decisions
Agent Fabric	Dynamic specialized workers
Tool Fabric	Controlled tool invocation
Experiment Engine	Hypothesis testing
Execution Sandbox	Safe code execution
Verification Engine	Tests and deterministic validation
Recovery Manager	Failure diagnosis and replanning

Module	Main Responsibility
Memory System	Project and experience memory
Policy Engine	Risk assessment and approvals
Git Manager	Branches, worktrees, diffs and commits
Observability	Traces, metrics and logs
Replay Engine	Historical execution reconstruction
Evaluation Engine	Benchmarks and experiments
Health Engine	Continuous software-quality analysis

28. Technology Stack

Layer	Proposed Technology
Desktop	Tauri 2
Frontend	React + TypeScript
Editor	Monaco Editor
Desktop Backend	Rust / Tauri APIs
Agent Runtime	Python
API/Runtime Interface	FastAPI
Agent Orchestration	LangGraph or custom stateful runtime
Data Validation	Pydantic
Repository Search	ripgrep
AST Analysis	Tree-sitter
Semantic Code Navigation	LSP
Vector Retrieval	Chroma
Local Persistent Database	SQLite
Optional Distributed State	Redis
Execution Isolation	Docker
Version Control	Git
Model Interface	LLM API Call

Layer	Proposed Technology
Telemetry	OpenTelemetry
Visualization	React-based dashboards
Packaging	Tauri bundling
Testing	Pytest + project-language-specific test frameworks
CI/CD	GitHub Actions

Tauri's current architecture is appropriate for a desktop-first product because it combines a web frontend with native capabilities through Rust and the system webview.

29. Hardware Requirements

29.1 Minimum Development Hardware

Component	Minimum
CPU	Intel Core i5 / AMD Ryzen 5 or equivalent
RAM	16 GB
Storage	512 GB SSD
GPU	Not required
Operating System	Windows 10/11, macOS, or Linux
Internet	Required for LLM API calls
Docker	Supported environment

29.2 Recommended Hardware

Component	Recommended
CPU	Intel Core i7 / AMD Ryzen 7 or better
RAM	32 GB
Storage	1 TB NVMe SSD
GPU	Optional NVIDIA GPU
VRAM	8 GB+ if local models are experimented with
Network	Stable broadband connection

A dedicated GPU is **not a mandatory requirement** because the LLM will initially be accessed using an **LLM API Call**. GPU resources become useful only if local inference or additional local AI components are introduced.

Docker-based execution also benefits from sufficient RAM and disk capacity because projects may require multiple isolated environments. Docker containers are intended to package software and its dependencies into isolated environments.

30. Software Requirements

30.1 Development Environment

- Python 3.11+
- Node.js LTS
- npm/pnpm
- Rust toolchain
- Git
- Docker Desktop / Docker Engine
- Tauri CLI
- Visual Studio Code or equivalent IDE
- Linux, Windows, or macOS development environment

Tauri requires Rust and platform-specific desktop development dependencies; for example, current Windows development requires Microsoft C++ Build Tools and WebView2, while macOS development uses Xcode or its command-line tools.

30.2 Core Libraries

Python:

- FastAPI
- Pydantic
- LangGraph
- httpx
- asyncio
- GitPython or subprocess-based Git integration
- pytest

Frontend:

- React
- TypeScript
- Monaco Editor
- Zustand/Redux
- React Router

- WebSocket client

Repository intelligence:

- Tree-sitter
- ripgrep
- LSP clients

Infrastructure:

- Docker
 - SQLite
 - Chroma
 - OpenTelemetry
-

31. Feasibility Study

31.1 Technical Feasibility

The project is technically feasible because its major components are already supported by mature open-source technologies.

Tauri provides desktop application capabilities, FastAPI provides the application/runtime interface, Tree-sitter provides incremental parsing, Docker provides containerized execution, SQLite provides local persistent storage, Chroma provides semantic retrieval, and OpenTelemetry provides observability infrastructure.

The principal technical difficulty is not implementing each individual technology but integrating them into a reliable autonomous execution system.

Conclusion: Technically feasible.

31.2 Economic Feasibility

The proposed system can be implemented primarily using open-source technologies.

Potential costs include:

- LLM API usage;
- optional hosted infrastructure;
- optional cloud databases;
- development hardware.

Because the model layer is represented as an **LLM API Call** rather than tied to a specific provider, the project can initially use an affordable or free development endpoint when available and later switch providers without modifying the core architecture.

Local databases and execution components substantially reduce mandatory infrastructure expenses.

Conclusion: Economically feasible for an academic project.

31.3 Operational Feasibility

The system is intended to operate as a desktop development environment.

A developer can:

1. Open a repository.
2. Describe a task.
3. Observe the agent's investigation.
4. Review proposed actions.
5. Approve risky actions.
6. Observe tests and verification.
7. Inspect the final diff.
8. Commit or export the result.

The interface therefore integrates with the normal software development workflow instead of requiring developers to leave their engineering environment.

Conclusion: Operationally feasible.

31.4 Security Feasibility

Autonomous software modification introduces significant security concerns.

The system must not allow arbitrary model-generated commands unrestricted access to the host.

Security mechanisms should include:

- Git worktrees;
- Docker sandboxing;
- filesystem scoping;
- command allowlists/denyls;
- network restrictions;
- resource limits;
- approval gates;

- audit logs;
- rollback;
- secret isolation.

Docker provides process-level isolation that can be used as one layer of this architecture, although containerization should not be treated as a complete security boundary by itself.

Conclusion: Feasible with layered security controls.

31.5 Schedule Feasibility

The complete vision is ambitious, but it can be implemented incrementally.

Phase 1 — Desktop Foundation

- Tauri
- React
- repository explorer
- editor
- terminal
- Git integration

Phase 2 — Repository Intelligence

- Tree-sitter
- ripgrep
- symbol extraction
- dependency graph
- project indexing

Phase 3 — Agent Runtime

- task interpretation
- planning
- tools
- LLM API Call
- execution loop

Phase 4 — Autonomous Engineering

- modifications
- tests
- debugging
- retries
- recovery
- worktrees

Phase 5 — Advanced Intelligence

- world model
- hypothesis engine
- experiment system
- adaptive worker allocation
- persistent memory

Phase 6 — Reliability and Governance

- risk scoring
- HITL
- observability
- replay
- evaluation

Phase 7 — Continuous Engineering

- technical-debt detection
- engineering-health engine
- autonomous maintenance queue

This staged approach makes the project achievable while allowing advanced capabilities to be added progressively.

32. Advantages of the Proposed System

The system differs from a conventional AI coding assistant in several ways.

Repository awareness

The system builds a persistent representation of the repository rather than depending exclusively on prompt context.

Adaptive autonomy

The system determines how much autonomy is appropriate rather than always behaving fully autonomously.

Dynamic specialization

Workers are generated or configured from capabilities instead of relying on a fixed collection of role-based agents.

Evidence-driven completion

Tests and deterministic execution results determine whether an action actually worked.

Failure recovery

The system can diagnose failures, restore checkpoints, and change strategies.

Long-term memory

Past engineering experiences can influence future work.

Risk-aware human control

Humans remain in control of high-risk actions.

Continuous engineering

The system can identify technical debt and software-health issues even when no explicit coding request is active.

33. Expected Outcomes

At the end of the project, the expected outcome is a working desktop application capable of:

1. Opening and indexing a software repository.
 2. Building a structured project representation.
 3. Accepting natural-language engineering tasks.
 4. Planning multi-step engineering activities.
 5. Dynamically allocating agent capabilities.
 6. Searching and understanding the repository.
 7. Modifying code.
 8. Creating isolated Git worktrees.
 9. Running tests and builds in controlled environments.
 10. Diagnosing failures.
 11. Performing iterative repair.
 12. Reviewing changes.
 13. Requesting human approval when required.
 14. Recording execution traces.
 15. Maintaining persistent project memory.
 16. Replaying previous engineering sessions.
 17. Generating engineering-health reports.
-

34. Evaluation Plan

The system should be evaluated using controlled software-engineering tasks rather than only subjective demonstrations.

34.1 Primary Metrics

Task Success Rate

Percentage of tasks completed successfully.

Test Pass Rate

Percentage of generated solutions for which required tests pass.

Regression Rate

Percentage of solutions that solve the target problem but break existing functionality.

Recovery Rate

Percentage of initially failed tasks that are successfully recovered without restarting from scratch.

Human Intervention Rate

Percentage of tasks requiring manual intervention.

Average Cost

LLM/API expenditure per completed task.

Latency

Time required to reach verified completion.

Tool Efficiency

Number of unnecessary tool calls.

Planning Efficiency

Difference between planned and actual execution path.

Reliability

Variance of results across repeated executions.

35. Experimental Comparison

The project can compare several system configurations:

```
Baseline A
Single LLM + direct coding

Baseline B
LLM + repository tools

Baseline C
LLM + tools + iterative verification

System D
Dynamic agent system

System E
Dynamic agents + World Model

System F
Full system
World Model + experiments + memory +
risk control + recovery
```

This allows the project to determine which architectural components actually contribute value.

This is important because Agentless demonstrates that additional agent complexity does not automatically imply better software-engineering performance.

36. Research Contribution

The academic contribution of the project is not expected to be a new foundation model.

Instead, the contribution lies in **system architecture and empirical investigation**.

Possible research questions include:

RQ1

Does a persistent Engineering World Model improve repository-level task completion compared with flat repository retrieval?

RQ2

Does adaptive agent formation outperform a fixed multi-agent architecture?

RQ3

Does experiment-driven debugging improve recovery from initial failures?

RQ4

Does persistent procedural memory reduce repeated failure patterns across engineering tasks?

RQ5

Can deterministic verification and risk-aware autonomy reduce unsafe or incorrect autonomous actions?

RQ6

What is the relationship between agent complexity, success rate, latency, and cost?

These questions provide a strong basis for extending the project into a research paper.

37. Limitations

Several limitations should be acknowledged.

First, the final capabilities will depend significantly on the selected LLM and its tool-calling and reasoning performance.

Second, fully autonomous software engineering cannot be guaranteed for arbitrary repositories.

Third, repository-level reasoning can become computationally expensive for very large codebases.

Fourth, sandboxing reduces risk but does not eliminate all security threats.

Fifth, model APIs may introduce latency, rate limits, availability constraints, or usage costs.

Sixth, agent evaluation remains an active research problem, and public benchmarks can contain dataset-quality or contamination issues. The 2026 discussion surrounding SWE-bench Verified demonstrates why benchmark results should be interpreted carefully.

38. Future Scope

Future versions can introduce:

- multi-repository reasoning;
- cloud execution workers;
- team collaboration;
- distributed agent scheduling;
- CI/CD integration;
- GitHub/GitLab integration;
- production incident response;
- autonomous documentation maintenance;
- automated pull-request management;
- project-level architecture optimization;
- software cost optimization;
- organization-wide engineering memory;
- learning-based agent routing;
- reinforcement-learning-based workflow optimization;
- multimodal debugging using screenshots and application state;
- local/private model execution.

A longer-term version could evolve from an AI coding environment into an **autonomous software organization capable of continuously maintaining a complete software product.**

39. Conclusion

The proposed Autonomous Software Engineering Operating System represents a shift from conventional AI-assisted programming toward persistent, evidence-driven autonomous software engineering.

Existing research has demonstrated the value of tool use, reasoning-action loops, multi-agent collaboration, repository-level coding, sandboxed execution, memory, and human oversight. However, these capabilities are frequently explored as separate components or narrow workflows.

The proposed system combines them around a different core abstraction: the **software project as a persistent world.**

The platform continuously models the project, determines what is known and unknown, chooses appropriate workers and tools, conducts experiments, performs changes in isolated environments, verifies outcomes using deterministic evidence, recovers from failures, remembers previous engineering experiences, and escalates consequential decisions to humans.

The project therefore goes beyond a conventional “multi-agent system.” Multi-agent execution becomes only one mechanism inside a broader architecture consisting of:

**Engineering World Model + Decision Engine + Dynamic Agent Fabric + Tool Fabric +
Experiment Engine + Verification + Recovery + Memory + Risk Governance + Continuous
Engineering Health**

The LLM remains replaceable and is represented architecturally as an **LLM API Call**, allowing the system to remain model-independent and adaptable as the LLM ecosystem evolves.

The resulting platform has both strong academic value and practical engineering relevance, while providing a substantial foundation for future research in autonomous software engineering, agent reliability, adaptive workflows, and human-agent collaboration.

40. References

- [1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing Reasoning and Acting in Language Models," 2022.
- [2] T. Schick et al., "Toolformer: Language Models Can Teach Themselves to Use Tools," 2023.
- [3] Q. Wu et al., "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation," 2023.
- [4] C. Qian et al., "ChatDev: Communicative Agents for Software Development," *Proceedings of ACL 2024*, pp. 15174–15186, 2024.
- [5] S. Hong et al., "MetaGPT: Meta Programming for Multi-Agent Collaborative Framework," 2023.
- [6] C. E. Jimenez et al., "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?" arXiv: 2310.06770, 2023/2024.
- [7] J. Yang et al., "SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering," arXiv: 2405.15793, 2024.
- [8] X. Wang et al., "OpenHands: An Open Platform for AI Software Developers as Generalist Agents," arXiv: 2407.16741, 2024.
- [9] K. Zhang et al., "CodeAgent: Enhancing Code Generation with Tool-Integrated Agent Systems for Real-World Repo-level Coding Challenges," *Proceedings of ACL 2024*, pp. 13643–13658, 2024.
- [10] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Agentless: Demystifying LLM-based Software Engineering Agents," arXiv:2407.01489, 2024.
- [11] D. Huang et al., "AgentCoder: Multi-Agent-based Code Generation with Iterative Testing and Optimisation," 2023.
- [12] J. S. Park et al., "Generative Agents: Interactive Simulacra of Human Behavior," arXiv:2304.03442, 2023.

- [13] H. Jin et al., "From LLMs to LLM-based Agents for Software Engineering: A Survey of Current, Challenges and Future," arXiv:2408.02479, 2024.
- [14] J. Liu et al., "Large Language Model-Based Agents for Software Engineering: A Survey," arXiv: 2409.02977, 2024.
- [15] Y. Wang et al., "Agents in Software Engineering: Survey, Landscape, and Vision," arXiv:2409.09030, 2024.
- [16] Y. Dong et al., "A Survey on Code Generation with LLM-based Agents," arXiv:2508.00083, 2025.
- [17] A. Yehudai et al., "A Survey on Evaluation of LLM-based Agents," *Findings of ACL 2026*, pp. 26690–26714, 2026.
- [18] OpenAI, "Introducing SWE-bench Verified," 2024, updated 2025.
- [19] OpenAI, "Why We No Longer Evaluate SWE-bench Verified," Feb. 2026.
- [20] LangChain, "LangGraph Overview and Persistence Documentation," 2026.
- [21] OpenTelemetry, "OpenTelemetry Documentation," 2026.
- [22] Tree-sitter, "Tree-sitter Documentation," 2026.
- [23] Docker, "Docker Documentation," 2026.
- [24] SQLite, "About SQLite," 2026.
- [25] Chroma, "Chroma Documentation," 2026.
- [26] Tauri, "Tauri 2 Documentation and Architecture," 2026.
- [27] FastAPI, "FastAPI Documentation," 2026.
- [28] PostgreSQL, "PostgreSQL Documentation: Transactions," 2026.